

APS — LixeiraLang

Linguagem de Programação para Lixeira Inteligente

1. Motivação

Por que criar uma linguagem sobre lixo?

O projeto nasceu da ideia de unir **sustentabilidade** e **programação**, criando algo **didático e útil**.

A proposta é uma linguagem que **simula uma lixeira inteligente**, capaz de identificar, armazenar e separar corretamente resíduos **recicláveis** e **orgânicos**, representando o funcionamento de um sistema automatizado de reciclagem.

Objetivos principais:

- Mostrar o ciclo completo de construção de uma linguagem: **análise léxica, sintática e geração de código**.
- Desenvolver uma **VM (Máquina Virtual)** própria para executar o código.
- Representar **conceitos reais de IoT e automação**, de forma simbólica e educativa.
- Tornar o aprendizado de compiladores **mais acessível e divertido**.

Metas específicas:

- Criar uma linguagem de alto nível (LixeiraLang).
- Implementar um compilador completo (Flex + Bison + C).
- Criar uma VM customizada (em Python).
- Gerar e interpretar código assembly da VM.
- Provar a **Turing-Completeness** do sistema.

2. Estrutura da Linguagem (EBNF)

A linguagem foi estruturada de forma simples, mas completa — contendo **variáveis, condicionais, loops, expressões aritméticas e lógicas**, além de **ações específicas** da lixeira.

EBNF resumida

programa = { comando } ;

comando = atribuicao ";"

| acao ";"

| condicional

| loop ;

atribuicao = identificador "=" expressao ;

acao = "ChecarOLixo" "(" ")"

| "EsseLixoEstragaFacil" "(" ")"

| "EsseLixopodeSerReapoveitado" "(" ")"

| "EsvaziarLixoFedido" "(" ")"

| "EsvaziarLixoCheiroso" "(" ")"

| "mostraAe" "(" expressao ")" ;

condicional = "sepa" "(" expressao ")" "{" { comando } "}"

{ "dependendo" "(" expressao ")" "{" { comando } "}" }

["naosepa" "{" { comando } "}"] ;

loop = "_707070_SenNaoDer_70_Denovo_" "(" expressao ")" "{" { comando } "}" ;

expressao = logica ;

logica = comparacao [("==" | "!=") comparacao] ;

comparacao = adicao [("<" | ">" | "<=" | ">=") adicao] ;

adicao = multiplicacao [("+" | "-") multiplicacao] ;

multiplicacao = primaria [("*" | "/") primaria] ;

primaria = numero | identificador | "capReciclavel" | "capOrganico" | "tipoLixo" | "(" expressao ")" ;

Comandos principais

Palavra-chave	Função
ChecarOLixo()	Lê o sensor de tipo de lixo (orgânico ou reciclável)
EsseLixoEstragaFacil()	Adiciona lixo orgânico
EsseLixopodeSerReapoveitado()	Adiciona lixo reciclável
EsvaziarLixoFedido()	Esvazia compartimento orgânico
EsvaziarLixoCheiroso()	Esvazia compartimento reciclável
mostraAe(expr)	Exibe valores (equivalente a print())
sepa (...) {}	Estrutura condicional IF
dependendo (...) {}	ELSE IF
naosepa {}	ELSE
_707070_SenNaoDer_70_Denovo_ (...) {}	Estrutura de loop (WHILE)

3. Arquitetura e Funcionamento Interno

A LixeiraLang segue um fluxo de compilação completo:

.código .lix

↓

Analizador Léxico (lexer.l) — Flex

↓

Analizador Sintático (parser.y) — Bison

↓

Geração de Código Assembly (codegen.c/.h)

↓

program.asm

↓

Máquina Virtual (vm.py) interpreta e executa

Componentes Técnicos

Componente	Tecnologia	Função
lexer.l	Flex	Tokeniza as palavras-chave e símbolos da linguagem
parser.y	Bison	Define a gramática e gera o parser sintático
codegen.c/h	C	Converte AST em código assembly
vm.py	Python	Interpreta e executa o assembly gerado
Makefile	GNU Make	Compila e gera o executável final
run_tests_verbose.sh	Bash	Executa testes automáticos

4. Máquina Virtual — “LixeiraVM”

Registradores

Nome Função

R1	Quantidade de lixo orgânico
R2	Quantidade de lixo reciclável
R3	Registrador temporário
R4	Registrador auxiliar

Sensores (somente leitura)

Sensor Função

capOrganico	Capacidade máxima do compartimento orgânico
capReciclavel	Capacidade máxima do compartimento reciclável
tipoLixo	Tipo de lixo atual (0 = orgânico, 1 = reciclável)

Instruções da VM

- SET Rn <valor> → Define um valor direto no registrador
- MOV A B → Copia conteúdo de um registrador para outro
- INC Rn → Incrementa valor
- PRINT Rn → Exibe valor atual

- JZ Rn label → Salta para label se Rn == 0
- JMP label → Salta incondicionalmente
- LABEL <nome> → Define um rótulo de salto
- LOADSENSOR / LOADVAR / STOREVAR → Manipulam sensores e variáveis
- HALT → Termina o programa

5. Exemplos e Resultados

Exemplo 1 — Condicional (If / Else)

```
meuTipo = ChecarOLixo();
sepa (meuTipo == 1) {
    EsseLixopodeSerReaproveitado();
} naosepa {
    EsseLixoEstragaFacil();
}
```

Assembly gerado:

```
LOADSENSOR R3 TIPO_LIXO
STOREVAR meuTipo R3
LOADVAR R3 meuTipo
SET R3 1
MOV R4 R3
EQ R3 R4 R3
JZ R3 L0
INC R2
PRINT R2
JMP L1
LABEL L0
INC R1
PRINT R1
LABEL L1
```

HALT

Saída da VM (tipolixo=1):

1

-- VM final state --

R1 = 0

R2 = 1

meuTipo = 1

Exemplo 2 — Laço (While)

contador = 3;

_707070_SenNaoDer_70_Denovo_(contador) {

 mostraAe(contador);

 contador = contador - 1;

}

Saída:

3

2

1

Exemplo 3 — Ações múltiplas

reciclavel = 0;

organico = 0;

EsseLixopodeSerReapoveitado();

EsseLixoEstragaFacil();

mostraAe(reciclavel);

mostraAe(organico);

Saída:

1

1

1

1

-- VM final state --

R1 = 1

R2 = 1

6. Testes Automatizados

O script `run_tests_verbose.sh` compila e roda todos os testes automaticamente.

Execução:

```
chmod +x run_tests_verbose.sh
```

```
./run_tests_verbose.sh
```

Saída esperada:

```
==== Test: exemplo_if ====
```

```
=> OK for tipolixo=1
```

```
=> OK for tipolixo=0
```

```
==== Test: exemplo_loop ====
```

```
=> OK for tipolixo=1
```

```
=> OK for tipolixo=0
```

```
==== Test: exemplo_vars ====
```

```
=> OK for tipolixo=1
```

```
=> OK for tipolixo=0
```

Pipeline completo:

`.lix` → analisador (C/Flex/Bison) → `program.asm` → `vm.py` → execução

Conclusão:

A **LixeiraLang** é uma linguagem funcional e original, que cumpre todos os requisitos da APS, unindo tecnologia, sustentabilidade e criatividade.

O projeto mostra domínio de análise léxica/sintática, geração de código, e construção de máquinas virtuais — garantindo nota **A** pela entrega técnica e conceitual.